

Algorithms

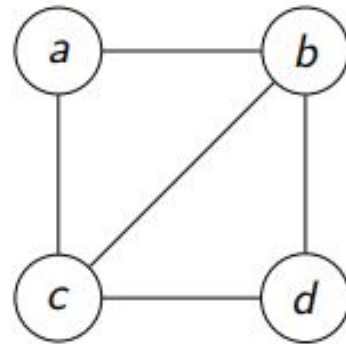
Lecture 1.2-2.1

Dr. Md Moniruzzaman

Graph

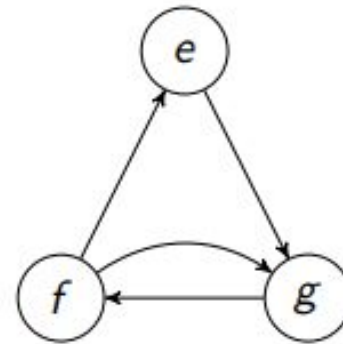
■ Graph, $G = (V, E)$

- V = Set of vertices/nodes → Denoted by circles
- E = Set of edges → Denoted by lines/arrows
 - ▶ Undirected: $e = \{u, v\} \rightarrow$ Unordered pairs
 - ▶ Directed: $e = (u, v) \rightarrow$ Ordered pairs



$$V = \{a, b, c, d\}$$

$$E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{c, d\}\}$$



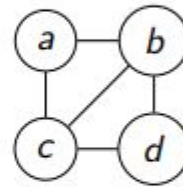
$$V = \{e, f, g\}$$

$$E = \{(e, g), (f, e), (f, g), (g, f)\}$$

Adjacency Matrix

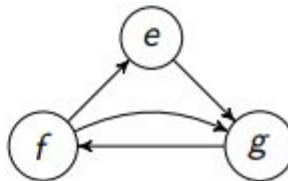
■ Adjacency Matrix

- Array $adjM$ of size $|V| \times |V|$
- Each element $\in \{0, 1\}$
- (Undirected) For vertices $u, v \in V$, $adjM[u][v] = \begin{cases} 1 & \text{if } \{u, v\} \in E \\ 0 & \text{o/w} \end{cases}$



	a	b	c	d
a	0	1	1	0
b	1	0	1	1
c	1	1	0	1
d	0	1	1	0

- (Directed) For vertices $u, v \in V$, $adjM[u][v] = \begin{cases} 1 & \text{if } (u, v) \in E \\ 0 & \text{o/w} \end{cases}$

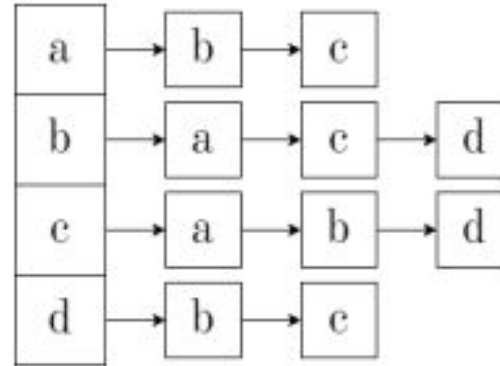
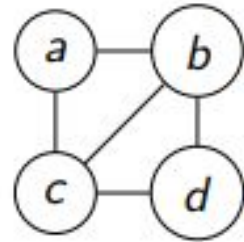


	e	f	g
e	0	0	1
f	1	0	1
g	0	1	0

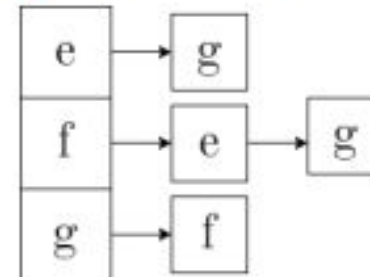
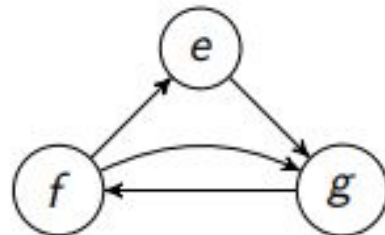
Adjacency List

■ Adjacency List

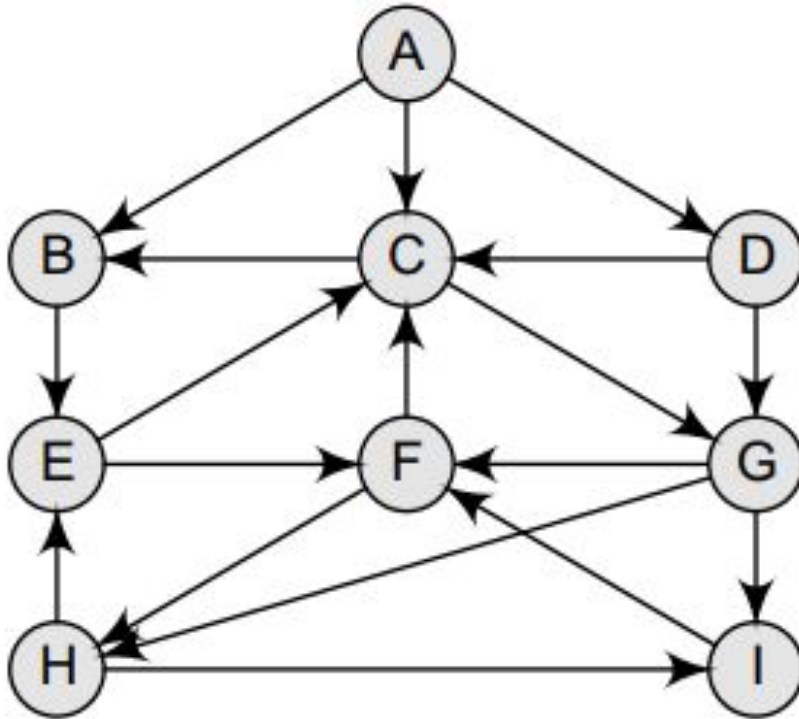
- Array `adjL` of size $|V|$
- Each element contains a linked list
- (Undirected) For each vertex $u \in V$, $adjL[u] = \{v \in V | \{u, v\} \in E\}$



- (Directed) For each vertex $u \in V$, $adjL[u] = \{v \in V | (u, v) \in E\}$



Sample directed graph



Adjacency lists

A: B, C, D

B: E

C: B, G

D: C, G

E: C, F

F: C, H

G: F, H, I

H: E, I

I: F

BFS (with 3 states)

Step 1: SET STATUS = 1 (ready state)
for each node in G

Step 2: Enqueue the starting node A
and set its STATUS = 2
(waiting state)

Step 3: Repeat Steps 4 and 5 until
QUEUE is empty

Step 4: Dequeue a node N. Process it
and set its STATUS = 3
(processed state).

Step 5: Enqueue all the neighbours of
N that are in the ready state
(whose STATUS = 1) and set
their STATUS = 2
(waiting state)
[END OF LOOP]

Step 6: EXIT

BFS (with 2 states)

- Initialize a **visited** array by marking every vertex as not-visited
- Declare a **queue** and insert the starting vertex.
- Mark the starting vertex as visited.
- Follow the below process till the queue becomes empty:
 - Remove the first vertex of the queue.
 - Insert all the unvisited neighbors of the vertex into the queue and mark those vertices as visited.

Implementation

- Using vector to store the graph as adjacency list:
 - **vector<int> lst[MAX];** //this is an array of vectors
 - To insert an element: **lst[index].push_back(value);**
 - To get the size of the vector: **int sz = lst[index].size();**
- To maintain a Queue:
 - **queue<int> Q;**
 - To insert an element: **Q.push(node);**
 - To retrieve the front element: **container = Q.front();**
 - To remove an element : **Q.pop();**
 - To check whether empty or not: **Q.empty();**

Adjacency List Implementation

```
void bfs(int node) {  
    queue<int> Q;  
    int temp;  
    Q.push(node);  
    visited[node] = 1;  
    while(!Q.empty()) {  
        temp = Q.front();  
        Q.pop();  
        int sz = lst[temp].size();  
        for(int k = 0; k < sz; k++) {  
            int nextnode = lst[temp][k];  
            if(!visited[nextnode]) {  
                visited[nextnode] = 1;  
                Q.push(nextnode);  
            }  
        }  
    }  
}
```

Adjacency Matrix Implementation

```
int bfs(int src) {  
    queue<int> Q;  
    int temp;  
    Q.push(src);  
    visited[src] = 1;  
    while(!Q.empty()) {  
        temp = Q.front();  
        Q.pop();  
        for(int i = 0; i < n; i++) {  
            if(g[temp][i] && !visited[i]) {  
                Q.push(i);  
                visited[i] = 1;  
            }  
        }  
    }  
    return 1;  
}
```

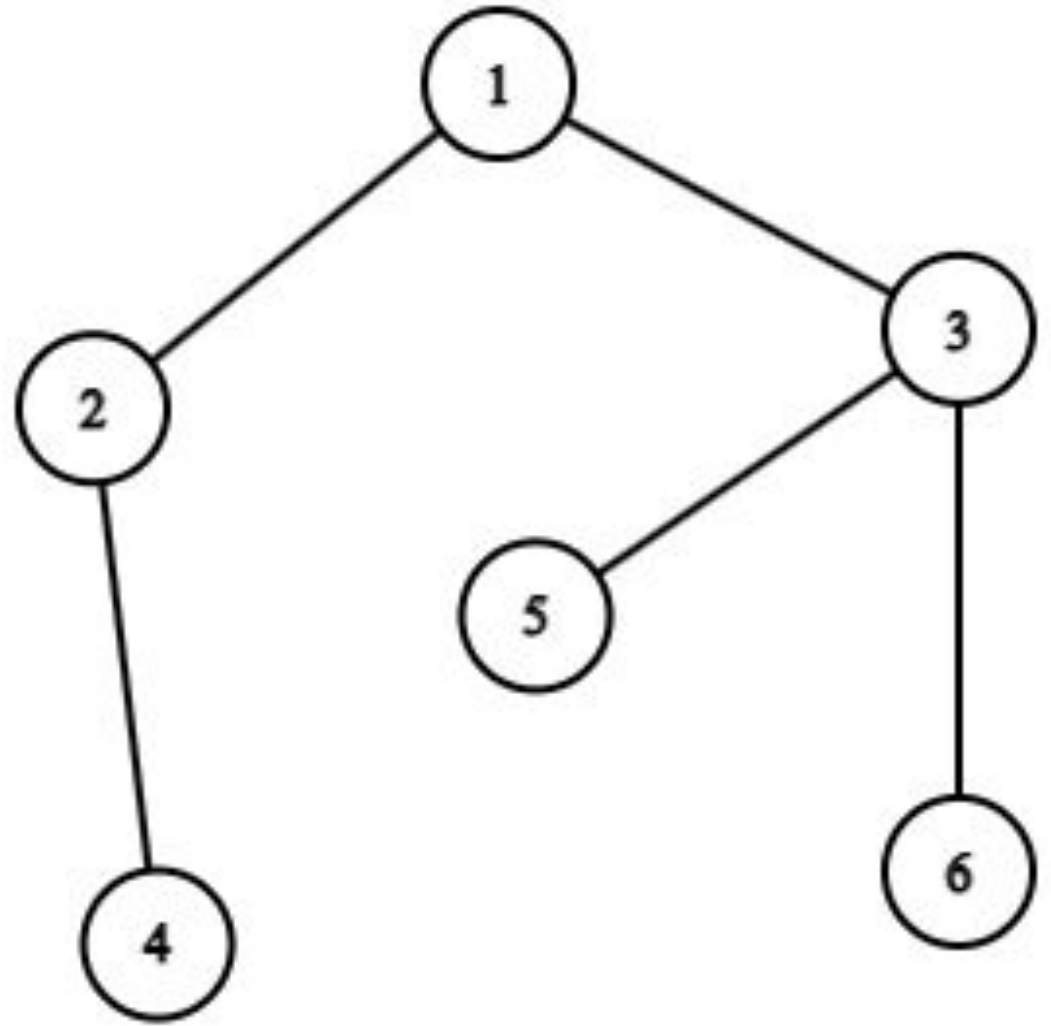
DFS (with 2 states)

- Initialize a **visited** array by marking every vertex as not-visited
- Declare a stack and insert the starting vertex.
- Mark the starting vertex as visited.
- Follow the below process till the stack becomes empty:
 - Remove the first vertex of the stack.
 - Insert all the unvisited neighbors of the vertex into the stack and mark those vertices as visited.

DFS (Continue...)

```
Step 1: SET STATUS = 1 (ready state) for each node in G
Step 2: Push the starting node A on the stack and set
        its STATUS = 2 (waiting state)
Step 3: Repeat Steps 4 and 5 until STACK is empty
Step 4:   Pop the top node N. Process it and set its
          STATUS = 3 (processed state)
Step 5:   Push on the stack all the neighbours of N that
          are in the ready state (whose STATUS = 1) and
          set their STATUS = 2 (waiting state)
          [END OF LOOP]
Step 6: EXIT
```

Graph to
traverse



Problem

- Equalizing Money (Lightoj)
 - <https://lightoj.com/problem/equalizing-money>
- Building Roads (CSES)
 - <https://cses.fi/problemset/task/1666/>